

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Inteligência Artificial Aplicada a Testes

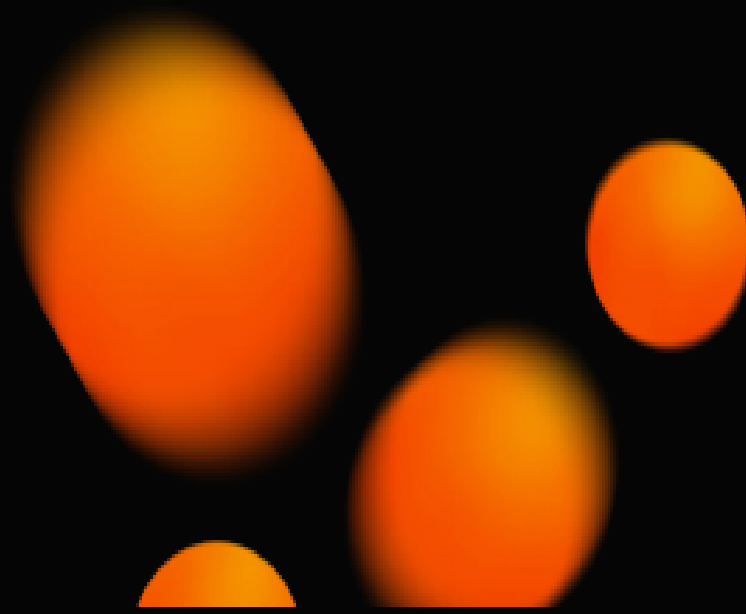
Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.

Kanoot!



Bora Revisar?





QA + IA em Escala e no Mercado

Inteligência Artificial Aplicada a Testes

Aula 3





Tendências atuais de IA em QA

Inteligência Artificial Aplicada a Testes

Aula 3



Principais tendências para QA

→ Automação Avançada e Agentes Autônomos

- ◆ Automação inteligente com agentes autônomos
- ◆ Predição de defeitos e autodiagnósticos.

→ IA Generativa e Testes Inteligentes

- ◆ Modelos generativos especializam-se em criar dados sintéticos e cenários realistas.
- ◆ Scripts de teste auto-regenerativos

→ Democratização e Plataformas No-Code

- ◆ Plataformas no-code e low-code permite que profissionais não técnicos participem dos processos de QA

→ Governança, Ética e Avaliação de Modelos

- ◆ Ferramentas de governança de IA são cruciais para validar questões de viés e cumprimento de normas (LGPD, GDPR).
- ◆ Testes éticos ganham espaço.

→ Integração com Big Data e Real-Time QA

- ◆ Análise de grandes volumes de dados de testes
- ◆ Dashboards inteligentes com dados em tempo real.

→ Testes Shift-Left e Shift-Right

- ◆ QA presente em todas as etapas do ciclo DevOps, da criação à produção.
- ◆ Automações no CI/CD com rollback automático em falhas.

→ Foco em Segurança e Auditoria Automatizada

- ◆ Monitoramento e proteção contra ameaças cibernéticas

**LATEST
NEWS**

Teste Visual

1. Snapshot Testing (tradicional)

- **Funcionamento:** tira uma foto da tela (ou HTML) e compara pixel a pixel com a versão anterior.
- **Problemas:**
 - ◆ Qualquer mudança mínima gera falso positivo (1px, sombra, cor de fonte).
 - ◆ Requer muita manutenção → flakiness visual.
- Ferramentas: Jest Snapshot, Selenium screenshots.

2. Visual AI Testing

- **Funcionamento:** a IA entende o layout e diferencia mudanças relevantes de cosméticas.
- **Exemplo prático:**
 - ◆ Botão mudou de azul para vermelho → IA marca como alteração significativa.
 - ◆ Texto moveu 1px para o lado → IA ignora.
- Ferramentas: Appliflow Eyes, Percy, LambdaTest Smart Testing.

"Enquanto snapshot é como um guarda que reclama até se você mudar de roupa, a IA visual é como um colega de QA que diz: 'Ok, mudou a cor do botão, mas isso realmente impacta a experiência do usuário?'"

Principais tendencias para QA

1. Snapshot Testing (tradicional)

→ **Funcionamento:** tira uma foto da tela (ou HTML) e compara pixel por pixel.

→ **Problemas:**

◆ Qual
som

◆ Requ

→ **Ferramentas:**

2. Visual AI Testing

→ **Funcionamento:** a IA entende o layout e diferencia as.

ermelho → IA marca

→ IA ignora.

laTest Smart Testing.



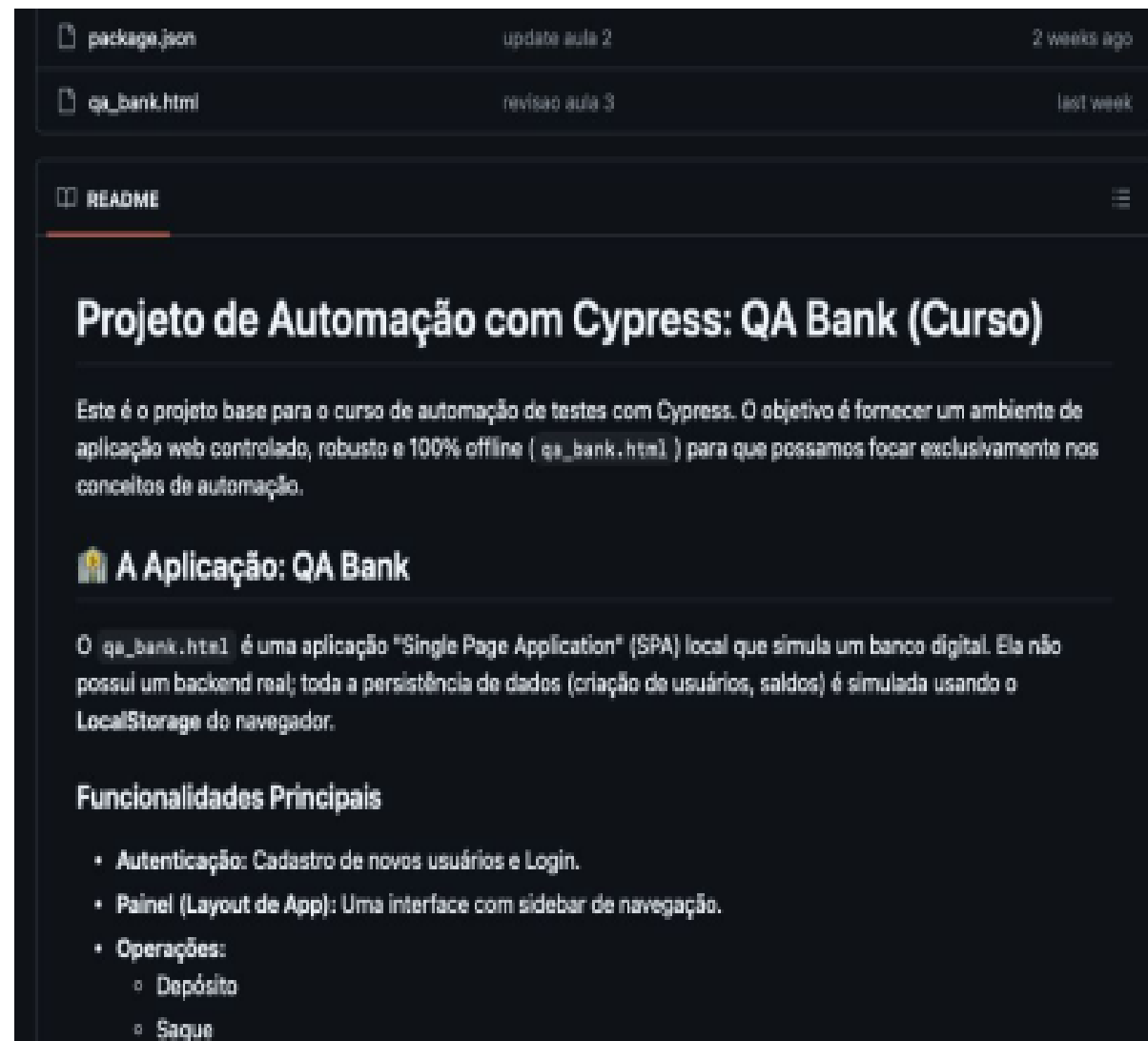
Steps

1. Clona o repositório git

Git clone

<https://github.com/joaopedrosantossilva/fast-automacaoweb-turma05>

2. Abre o projeto em uma IDE de sua preferência



Instalação: O Setup do Projeto

Passo	Comando (Terminal)	O que faz? (A Explicação)
1	<code>npm init -y</code>	Cria o Projeto: Gera o arquivo <code>package.json</code> , que é a "certidão de nascimento" do nosso projeto Node.js.
2	<code>npm install cypress --save-dev</code>	Instala o Cypress: Baixa o Cypress e o salva como uma "Dependência de Desenvolvimento" no <code>package.json</code> .
3	<code>npx cypress open</code>	Executa o Cypress: Abre o Test Runner , a interface visual onde podemos ver, rodar e depurar nossos testes em tempo real.

<https://docs.cypress.io/app/get-started/install-cypress>

Demo AppliTools Eyes + Cypress

 Snapshot x  Visual AI

1. Abra o VS Code ou IDE de sua preferência
 - a. Crie uma pasta e abra um terminal nessa mesma pasta. Digite o comando para iniciar o projeto na pasta.

```
bash
```

```
npm init -y
```

Instalar dependências:

```
bash
```

```
npm i -D cypress @simonsmith/cypress-image-snapshot @appliTools/eyes-cypress
```

Rodar uma vez o Cypress:

```
bash
```

```
npx cypress open
```

Demo AppliTools Eyes + Cypress

 Snapshot x  Visual AI

2. Configuração Básica

- `cypress.config.js` → plugins de snapshot e AppliTools
- `support/e2e.js` → importar comandos

Exemplo:

```
js
import '@applitools/eyes-cypress/commands';
```

3. Configurar AppliTools

Windows PowerShell

```
powershell
$env:APPLITOLS_API_KEY="SEU_TOKEN"
```

1. Criar conta no applitools.com ➤
2. Copiar **API Key**
3. Exportar no terminal:

```
bash
export APPLITOLS_API_KEY=COLE_SUA_CHAVE
```

4. (Opcional) Criar `applitools.config.js` com `appName`, `batch` e `browsers`

4. Configurar o plugin de snapshot: [cypress.config.js](#) (na raiz do projeto)

JS cypress.config.js > ...

```
1  const { defineConfig } = require('cypress');
2  const eyesPlugin = require('@applitools/eyes-cypress');
3  const { addMatchImageSnapshotPlugin } = require('@simonsmith/cypress-image-snapshot/plugin');
4
5  module.exports = eyesPlugin(defineConfig({
6    e2e: {
7      setupNodeEvents(on, config) {
8        addMatchImageSnapshotPlugin(on, config);
9        return config;
10      },
11      baseUrl: 'https://applitools.com',
12    },
13  }));
```

cypress/support/e2e.js

Adicione estas linhas (no topo do arquivo):

```
import { addMatchImageSnapshotCommand } from '@simonsmith/cypress-image-snapshot/command';

addMatchImageSnapshotCommand(); // cria cy.matchImageSnapshot(...)
```

Demo AppliTools Eyes + Cypress

 Snapshot x  Visual AI

5. Crie/Importe os arquivos de teste

```
▼ e2e
  JS 01-snapshot.cy.js
  JS 02-snapshot.cy.js
```

6. Rode os testes

1. `npx cypress run --spec cypress/e2e/01-snapshot.cy.js`
2. `npx cypress run --spec cypress/e2e/02-snapshot.cy.js`

Demo AppliTools Eyes + Cypress

🔧 Troubleshooting (os mais comuns)

- Erro "failCypressAfterAllSpecs" → o plugin não foi registrado.
Confirme o `require('@appliTools/eyes-cypress')(on, config);` no `cypress.config.js` e retorne `config`.
- Nada aparece no dashboard → faltou exportar `APPLITOLS_API_KEY` no mesmo terminal do `cypress run`.
- Porta ocupada → troque porta: `npx serve v1 -l 8080` e ajuste `baseUrl` no `cypress.config.js`.
- SDK/versões → use `cypress@12.17.4` + `@appliTools/eyes-cypress` atual.
Se der cache estranho: `rm -rf node_modules package-lock.json && npm i`.

Test Flakiness + Self-healing

- Test Flakiness refere-se à natureza instável e imprevisível de um teste automatizado, que passa ou falha sem qualquer alteração no código ou no ambiente de execução.
- Teste que falha não porque o sistema está errado, mas porque **o teste em si é instável**.
- **Causas comuns:**
 - Timing issues: página ainda carregando, animações, elementos assíncronos.
 - Seletores frágeis: ID/CSS/XPath que muda a cada deploy.
 - Dependências externas: API instável, banco de dados compartilhado.
 - Ambiente: diferenças entre browsers, SO, rede.



Vamos praticar!

1. Abra o Google Colab → File > Upload notebook → selecione o [ML_Test_Prioritization_Demo.ipynb](#)
2. Rode as células na ordem.
3. Quando pedir, faça upload do [test_prioritization_summary.csv](#)
4. O notebook já:
 - instala e importa dependências,
 - treina Regressão e Classificação,
 - mostra importâncias/SHAP,
 - gera o ranking ([test_priority_ranking.csv](#) e [test_priority_ranking_classification.csv](#)).





Dica:
Plano de Estudos
no drive

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados